Practical: Plant Disease Detection using Convolutional Neural Network (CNN)

Laboratory Practice V – Computer Engineering

1 Aim

Implement a plant disease detection and classification system using Convolutional Neural Network (CNN). Use the Plant Disease Dataset containing images of healthy and diseased plant leaves.

1.1 Objective

Students should be able to build, train, and evaluate a CNN model for image classification to detect plant diseases from leaf images.

2 Theory

2.1 What is Image Classification?

Image classification is a task where the model looks at an image and tells which **category** it belongs to. In this practical, the model looks at a leaf image and tells if the plant is **healthy or diseased** and what type of disease it has.

2.2 What is CNN (Convolutional Neural Network)?

A CNN is a type of Deep Neural Network specially designed for **image processing**. It automatically learns features like edges, textures, and patterns from images.

2.2.1 Key Layers in CNN

Layer	Purpose
Conv2D	Applies filters to extract features (edges, patterns) from the image
MaxPooling2D	Reduces the size of feature maps by taking the maximum value in each region
Flatten	Converts 2D feature maps into a 1D vector
Dense	Fully connected layer for classification
Dropout	Randomly disables neurons during training to prevent overfitting
Softmax	Converts output to probabilities (for multi-class classification)

2.3 CNN Architecture Used

```
Input Image (128x128x3)
      |
Conv2D(32 filters, 3x3) + ReLU --> Feature Extraction
      |
MaxPooling2D(2x2) --> Size Reduction
      |
Conv2D(64 filters, 3x3) + ReLU --> More Features
      |
MaxPooling2D(2x2) --> Size Reduction
      |
Conv2D(128 filters, 3x3) + ReLU --> Complex Features
      |
MaxPooling2D(2x2) --> Size Reduction
      |
Flatten --> Convert to 1D
      |
Dense(128) + ReLU --> Classification
      |
Dropout(0.5) --> Prevent Overfitting
      |
Dense(10) + Softmax --> 10 Disease Classes
```

2.4 Dataset

The Plant Disease Dataset contains images organized in folders:

- **train/** — 18,504 images for training (10 classes)
- **valid/** — 4,626 images for validation (10 classes)
- **test/** — for testing

Each subfolder name represents a class (e.g., "Apple_Black_rot", "Tomato_healthy", etc.).

2.5 Data Augmentation

To make the model more robust, training images are randomly transformed:

- **Rotation** (up to 20 degrees)
- **Zoom** (up to 20%)
- **Horizontal Flip**
- **Rescaling** (pixel values 0–255 $\rightarrow$ 0–1)

Validation and test data are only rescaled (not augmented).

3 Code

Listing 1: Plant Disease Detection using CNN

```
# Cell 1: Import Libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
   Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt

# Cell 2: Set Dataset Paths
train_path = "Plant_Disease_Dataset/train"
valid_path = "Plant_Disease_Dataset/valid"
test_path  = "Plant_Disease_Dataset/test"

# Cell 3: Data Augmentation & Preprocessing
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

valid_datagen = ImageDataGenerator(rescale=1./255)
test_datagen  = ImageDataGenerator(rescale=1./255)

# Cell 4: Load Images from Directories
train_data = train_datagen.flow_from_directory(
    train_path,
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical'
)

valid_data = valid_datagen.flow_from_directory(
    valid_path,
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical'
)

test_data = test_datagen.flow_from_directory(
    test_path,
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical'
)

# Cell 5: Build CNN Model
model = Sequential()

# Conv Layer 1
```

```
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)
    ))
model.add(MaxPooling2D(2,2))

# Conv Layer 2
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

# Conv Layer 3
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

# Flatten
model.add(Flatten())

# Fully Connected
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

# Output Layer
model.add(Dense(train_data.num_classes, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# Cell 6: Train Model
history = model.fit(
    train_data,
    epochs=3,
    validation_data=valid_data
)

# Cell 7: Plot Accuracy Graph
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("CNN Accuracy Graph")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

4 Output

4.1 Model Summary

```
Model: "sequential"
+-----------------------+------------------+-----------+
```

```
| Layer (type)              | Output Shape         | Param #    |
+---------------------------+----------------------+------------+
| conv2d (Conv2D)           | (None, 126,126,32)|        896 |
| max_pooling2d             | (None, 63, 63, 32)|          0 |
| conv2d_1 (Conv2D)         | (None, 61, 61, 64)|     18,496 |
| max_pooling2d_1           | (None, 30, 30, 64)|          0 |
| conv2d_2 (Conv2D)         | (None, 28, 28,128)|     73,856 |
| max_pooling2d_2           | (None, 14, 14,128)|          0 |
| flatten (Flatten)         | (None, 25088)      |          0 |
| dense (Dense)             | (None, 128)        | 3,211,392 |
| dropout (Dropout)         | (None, 128)        |          0 |
| dense_1 (Dense)           | (None, 10)         |      1,290 |
+---------------------------+----------------------+------------+
Total params: 3,305,930  (12.61 MB)
```

4.2 Training Output

```
Epoch 1/3 - accuracy: 0.6051 - loss: 1.1220
            val_accuracy: 0.8042 - val_loss: 0.5185
Epoch 2/3 - accuracy: 0.7819 - loss: 0.6007
            val_accuracy: 0.8982 - val_loss: 0.3054
Epoch 3/3 - accuracy: 0.8576 - loss: 0.4184
            val_accuracy: 0.9460 - val_loss: 0.1686
```

Model ne sirf 3 epochs mein 85% training accuracy aur 94.6% validation accuracy achieve ki!

5 Simple Explanation (Hinglish)

5.1 Is Practical Ka Aim Kya Hai?

Humein ek CNN model banana hai jo **plant ki leaf (patti) ki photo** dekhke bata sake ki plant **healthy hai ya bimaar** hai, aur agar bimaar hai toh **kaunsi bimari** hai. Dataset mein 10 alag-alag classes hain (different diseases + healthy).

Yeh ek **Image Classification problem** hai — matlab image dekhke category batani hai.

5.2 Karna Kya Hai? (Step by Step — Har Cell Ka Matlab)

5.2.1 Cell 1: Import Libraries

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
   Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
```

- `tensorflow/keras` — neural network banana, train karna
- `Conv2D` — image se features (patterns) nikalne wali layer

- `MaxPooling2D` — image ki size chhoti karne wali layer
- `Flatten` — 2D data ko 1D mein convert karna
- `Dense` — normal fully connected layer
- `Dropout` — overfitting rokne ke liye randomly neurons band karta hai
- `ImageDataGenerator` — images load karna aur augment karna
- `matplotlib` — graphs banana

Simple: Sab zaruri tools import kiye.

5.2.2 Cell 2: Dataset Paths

```
train_path = "Plant_Disease_Dataset/train"
valid_path = "Plant_Disease_Dataset/valid"
test_path  = "Plant_Disease_Dataset/test"
```

Teeno folders ka path set kiya: training images, validation images, aur test images.

Simple: Model ko bataya ki images kahan hain.

5.2.3 Cell 3: Data Augmentation

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)
valid_datagen = ImageDataGenerator(rescale=1./255)
test_datagen  = ImageDataGenerator(rescale=1./255)
```

- `rescale=1./255` — pixel values ko 0–255 se 0–1 range mein laata hai (normalize)
- `rotation_range=20` — image ko randomly 20 degrees tak ghuma deta hai
- `zoom_range=0.2` — image ko randomly 20% tak zoom karta hai
- `horizontal_flip=True` — image ko randomly horizontally flip karta hai

Kyun? Augmentation se model zyada robust banta hai — thoda ghumi hui ya zoomed image bhi pehchaan lega. Validation aur test pe augmentation nahi lagaate — sirf rescale karte hain.

Simple: Training images ko thoda modify karke variety bana rahe hain taaki model zyada accha seekhe.

5.2.4 Cell 4: Load Images from Folders

```
train_data = train_datagen.flow_from_directory(
    train_path, target_size=(128,128),
    batch_size=32, class_mode='categorical')
```

- `flow_from_directory` — folder structure se automatically images aur labels load karta hai
- `target_size=(128,128)` — sabhi images ko 128×128 pixels mein resize karta hai
- `batch_size=32` — ek baar mein 32 images process karega
- `class_mode='categorical'` — 10 classes ke liye one-hot encoding use karega
- Output: "Found 18504 images belonging to 10 classes"

Simple: Folders se images uthake model ke liye ready kiye. Har subfolder ek class hai.

5.2.5 Cell 5: Build CNN Model

```
model = Sequential()
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)
   ))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))
```

Layer by layer samjho:

1. **Conv2D(32, 3×3)** — 32 filters lagake image se basic features (edges, lines) nikalte hain
2. **MaxPooling2D(2×2)** — image ki size half karti hai (128→63)
3. **Conv2D(64, 3×3)** — 64 filters se zyada complex features nikalte hain
4. **MaxPooling2D** — size phir se half (63→30)
5. **Conv2D(128, 3×3)** — 128 filters se advanced features (disease patterns) nikalte hain
6. **MaxPooling2D** — size phir se half (30→14)
7. **Flatten** — 14×14×128 = 25,088 values ko ek line mein rakh diya
8. **Dense(128, relu)** — 128 neurons ki fully connected layer — classification ke liye
9. **Dropout(0.5)** — training mein 50% neurons randomly band — overfitting rokta hai
10. **Dense(10, softmax)** — 10 classes ke liye output — har class ki probability deta hai

Compile:

- `adam` — optimizer jo weights adjust karta hai
- `categorical_crossentropy` — multi-class classification ke liye loss function
- `accuracy` — metric jisse track karein ki model kitna sahi predict kar raha hai

Total parameters: 33 lakh+ (33,05,930) — yeh sab weights hain jo model training mein seekhta hai.

Simple: Image → features nikalo → size chhoti karo → 3 baar repeat → flatten → classify → 10 mein se kaunsa disease hai batao.

5.2.6 Cell 6: Train Model

```
history = model.fit(
    train_data, epochs=3,
    validation_data=valid_data)
```

- `epochs=3` — poora dataset 3 baar dekhega (zyada epochs = zyada accuracy)
- `train_data` — training images pe seekhega
- `validation_data` — har epoch ke baad validation pe check karega
- Epoch 1: 60% accuracy → Epoch 3: 85% training, 94.6% validation!

Simple: Model ne 18,000+ images dekhke seekha ki kaunsi leaf healthy hai aur kaunsi bimaar.

5.2.7 Cell 7: Plot Accuracy Graph

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("CNN Accuracy Graph")
plt.show()
```

Dono lines (training + validation) **upar jaani chahiye** — matlab model improve ho raha hai. Agar validation line neeche jaane lage toh **overfitting** ho rahi hai.

Simple: Graph se dekhte hain ki model time ke saath accha ho raha hai ya nahi.

5.3 Final Outcome Kya Milta Hai?

1. **Trained CNN Model** — jo leaf ki photo dekhke disease detect kar sakta hai
2. **94.6% Validation Accuracy** — sirf 3 epochs mein!
3. **Accuracy Graph** — model kaise improve hua over epochs
4. **10-class classifier** — 10 different disease/healthy categories identify kar sakta hai

Bottom Line: Is practical mein humne dekha ki CNN kaise images se automatically features seekhta hai (edges, textures, patterns) aur unke basis pe classify karta hai. Yeh Deep Learning ka sabse powerful application hai — Image Classification. CNN manually feature engineering ki zarurat khatam kar deta hai!

6 How to Run

6.1 On Google Colab

1. Go to `colab.research.google.com`
2. Upload the `.ipynb` notebook
3. Upload/Mount the `Plant_Disease_Dataset` folder
4. Runtime → Change runtime type → **T4 GPU** (faster training)
5. Run all cells

6.2 On Fresh Ubuntu PC

1. **Update system:**

```
sudo apt update && sudo apt upgrade -y
```

2. **Install Python 3 and pip:**

```
sudo apt install python3 python3-pip python3-venv -y
```

3. **Create virtual environment:**

```
python3 -m venv dl_env
source dl_env/bin/activate
```

4. **Install libraries:**

```
pip install tensorflow matplotlib jupyter
```

5. **Copy dataset and notebook to project folder:**

```
mkdir ~/PlantDisease && cd ~/PlantDisease
# Copy Plant_Disease_Dataset folder and .ipynb file here
```

6. **Run:**

```
jupyter notebook
# Open .ipynb file and run all cells
```

7 Conclusion

In this practical, we implemented Plant Disease Detection using a Convolutional Neural Network (CNN). The model architecture consisted of 3 convolutional layers with increasing filters (32, 64, 128) followed by max pooling layers, a flatten layer, a dense layer with dropout, and a softmax output layer for 10-class classification. Data augmentation techniques including rotation, zoom, and horizontal flipping were applied to improve model generalization. The model achieved approximately 85% training accuracy and 94.6% validation accuracy in just 3 epochs, demonstrating the effectiveness of CNNs for image classification tasks.